

HiReKG: High-Recall Knowledge Graph Extraction from Plain Text with Language Models

Anonymous ACL submission

Abstract

Knowledge graphs (KGs) represent information as an interlinked network of entities, capturing the relational context that remains implicit in unstructured text. Most existing LLM-based text to KG converters follow an *entity-first* pipeline: entities are extracted upfront, and relations are derived only between them. This design favors precision but systematically misses facts about a single entity, capping recall. We propose **HiReKG**, a high-recall extraction framework that reduces reliance on upfront entity extraction and introduces an object resolution module to keep the resulting triples cohesive. We evaluate HiReKG across three diverse datasets spanning general, scientific, and technical domains, using three LLM backbones. HiReKG improves recall over the strongest baseline by 18 percentage points on average (up to 29), with a precision trade-off of 2–9 points (5.7 on average), and recovers the single-entity facts that entity-first pipelines structurally miss.

1 Introduction

Knowledge graphs (KGs) provide a structured representation of entities and the relations between them, enabling explicit multi-hop traversal that unstructured text alone does not support. Integrating KGs into retrieval-augmented generation and recommendation systems has been shown to substantially improve their ability to consolidate scattered context from source data (Ma et al., 2025; Chen et al., 2025; Yu et al., 2022). These benefits have motivated a long line of work on automatic text-to-KG extraction, from classical OpenIE (Angeli et al., 2015) to recent LLM-based approaches (Bian, 2025), all aimed at converting unstructured text into structured graphs that can be used by downstream NLP tasks.

Several LLM-based methods have been proposed for text-to-KG extraction, including KGGen (Mo et al., 2025), RAKG (Zhang et al., 2025),

GraphRAG (Edge et al., 2025), PiVe (Han et al., 2024), and iText2KG (Lairgi et al., 2024). Despite differences in their specific designs, all of these methods share a common pattern: entities are first extracted from the source text, and relations are then derived only between pairs of those extracted entities. We refer to this shared pattern as the *entity-first* pipeline.

The *entity-first* design favors precision because every relation is grounded against an explicit entity set, but it carries a structural consequence. Any entity missed in the first stage is lost entirely, since no relation involving it can be recovered later. Equally, any fact in source text that describes a single entity in isolation, without a second entity to pair with, falls outside the relation-extraction step by construction. Together, these two failure modes produce a systematic coverage gap, particularly on facts that involve a single entity. To address this gap, we propose **HiReKG**, a high-recall text-to-KG extraction framework that reduces reliance on upfront entity extraction, allowing the LLM to surface facts about each entity directly from the source text, including those that do not pair with a second entity. However, removing the entity-pair constraint can leave the graph disconnected and repetitive, with the same entity scattered across multiple unlinked mentions. HiReKG resolves this with an object resolution module that restores connectivity by linking these mentions into shared graph nodes.

The main contributions of this paper are as follows:

- **HiReKG**, a high-recall text-to-KG extraction framework that, unlike the dominant entity-first paradigm, recovers single-entity facts while preserving competitive precision.
- A single-entity vs. multi-entity decomposition showing that entity-first baselines systematically under-recover single-entity facts, a gap that HiReKG largely closes.

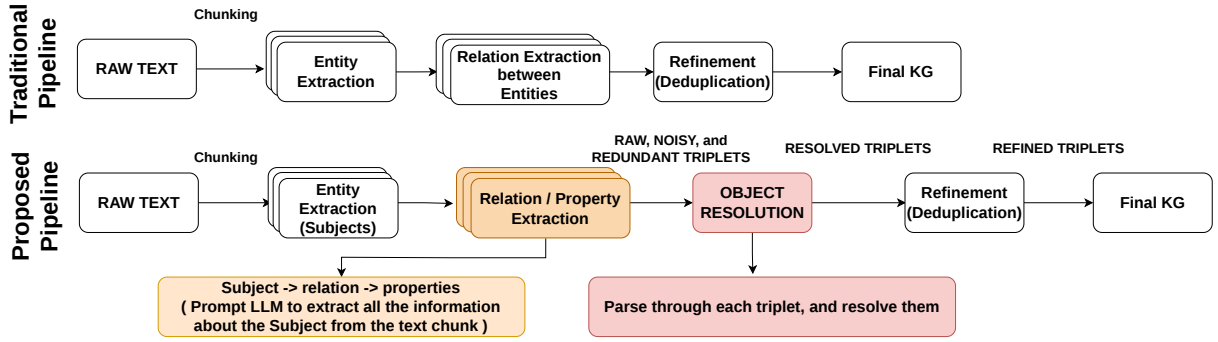


Figure 1: Comparison of the entity-first paradigm (top) and proposed **HiReKG** (bottom).

2 HiReKG

HiReKG extracts a knowledge graph from plain text in three stages: subject extraction, property extraction, and object resolution, followed by the refinement stage of Mo et al. (2025). The source document is first split into chunks, and each stage runs over all chunks before the next begins. Figure 1 contrasts HiReKG with the entity-first paradigm: subject extraction mirrors the entity-extraction step of prior pipelines, but HiReKG then decouples property extraction from the entity-pair anchor and adds an object resolution step that restores graph connectivity before refinement. A full end-to-end run on a sample paragraph is provided in Appendix A.

2.1 Subject Extraction

Following prior LLM-based extractors (Mo et al., 2025; Zhang et al., 2025), we prompt an LLM on each chunk to identify the set of entities mentioned in the source document. Each entity in this set serves as a *subject* around which information is later organized.

2.2 Property Extraction

For each chunk, the LLM is prompted to gather all information about the set of subjects identified in Section 2.1. The output is a set of *triples* (subject, relation, object), where the subject is one of the extracted entities, the relation is a free-form predicate, and the object is a natural-language phrase describing a property, attribute, or related entity. Crucially, and unlike entity-first methods, the object is not constrained to be another extracted entity: it may be a date, a quantity, a descriptive clause, or a phrase that itself references one or more entities.

This relaxation is what lets HiReKG recover single-entity facts. A statement such as “The Eiffel

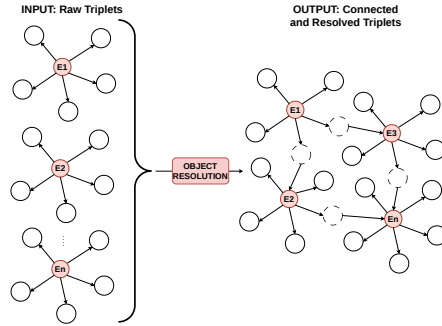


Figure 2: Object resolution links per-subject triples (left) by merging coreferent objects into shared nodes (right).

Tower is 330 metres tall” yields the triple (Eiffel Tower, height, 330 metres) keyed on a single subject, with no second entity required.

The trade-off is structural. When an object is a phrase that references other entities, those entities remain trapped inside the object string rather than appearing as proper graph nodes; for instance, “Marie Curie won the Nobel Prize in Physics and the Nobel Prize in Chemistry” is extracted as a single triple whose object embeds two prize entities that should each be a node in their own right. As a result, the graph initially fragments into per-subject *stars*, as shown in the left pane of Figure 2. Object resolution repairs this fragmentation.

2.3 Object Resolution

Object resolution rewrites raw triples so that any extracted entity appearing inside an object string becomes a proper object (Figure 2). For each raw triple, the object is normalized (lower-cased, lemmatized, with determiners stripped) and matched against the extracted entity set using word-boundary string matching; triples with no match are forwarded unchanged. Triples with one embedded entity can be split apart by a fixed rule, while triples with multiple embedded entities can

be split in several different ways and need the LLM to choose. We therefore use two repair procedures.

Single embedded entity. The object is tokenized and decomposed into a *prefix* (text before the entity), the entity span, and a *suffix* (text after the entity). In the common case, only a prefix is present: the prefix is folded into the relation and the entity becomes the new object, producing (s, r', e) where r' is the relation with the prefix appended. This deterministic path handles the majority of single-entity cases without an LLM call. When a suffix is present, an LLM rewrites the triple so that the entity becomes the object and any remaining content is folded into the relation.

Multiple embedded entities. The LLM is prompted with the original triple, the list of detected entities, and a structured analysis of each entity’s surrounding context. It selects one of three decomposition strategies and emits a set of triples in which every embedded entity appears as a proper object. HUB handles parallel lists, such as the two Nobel-Prize triple from Section 2.2; CHAIN handles sequential connectives; and NESTED handles hierarchical phrasing.

All LLM-rewritten triples are validated for entity coverage and field completeness, reverting to the original on failure. By promoting embedded entities to graph nodes, object resolution converts the per-subject *stars* of Section 2.2 into a connected graph without imposing the entity-first constraint at extraction time. Worked examples for each pattern (single-entity, HUB, CHAIN, NESTED) appear in Appendix B.

3 Experiments

3.1 Datasets

We evaluate on three datasets spanning distinct text styles. **MINE-1** (Mo et al., 2025) provides 100 LLM-generated documents paired with manually verified atomic facts that serve as a recall ground truth. **SciERC** (Luan et al., 2018) contains 100 scientific abstracts annotated with entities and relations, representing dense technical prose. **Re-DocRED** (Tan et al., 2022) is a cleaned version of DocRED (Yao et al., 2019) built from Wikipedia documents; we use the 15-document subset introduced by Sun et al. (2025). All three datasets have been used in recent text-to-KG work (Huang et al., 2025; Zhang et al., 2025), enabling direct comparison with prior results.

3.2 Baselines and Metrics

Baselines. We compare HiReKG against **KGGen** (Mo et al., 2025) and **RAKG** (Zhang et al., 2025), the two strongest representatives of the entity-first paradigm.

Metrics. We report recall and precision via two LLM-as-judge metrics. **MINE Score** (Mo et al., 2025) (recall) is the ratio of ground truth atomic facts that can be inferred from the extracted graph. **Faithfulness Score** (precision), adapted from RAG evaluation (Es et al., 2024), measures the fraction of extracted triples supported by the source text. As a supporting measure, the deterministic **Unique Information Ratio (UIR)** captures redundancy as the fraction of non-overlapping triples; UIR results are reported in Appendix E. Full prompts and configurations of all metrics are in Appendix C.

Models. We run each method with three LLM backbones: **GPT-4o** (OpenAI, 2024), **Qwen3-14B** (Qwen Team, 2025a), and **Qwen3-8B** (Qwen Team, 2024), spanning one flagship closed model and two open-weight models of different scales. Exact versions and hyperparameters are listed in Appendix D.

3.3 Single-Entity Decomposition Results

Our central hypothesis is that entity-first pipelines structurally under-recover facts that describe a single entity in isolation. To test this directly, we classify each atomic fact in the ground truth as *single-entity* (mentioning one of the document’s entities) or *multi-entity* (connecting two or more), and report the MINE Score on each subset.¹

Table 1 reports the decomposition, and the pattern is consistent with the hypothesis. KGen and RAKG show a systematic single-entity deficit: their single-entity score is lower than their multi-entity score in 14 of 18 configurations, with average gaps of -7.4 pp and -11.0 pp respectively. The deficit is sharpest on Re-DocRED, where both baselines collapse to single-entity scores of 35.7% and 37.5% with GPT-4o while remaining above 53% on multi-entity facts. HiReKG, in contrast, is near-symmetric (average gap -2.4 pp), and its absolute single-entity score exceeds the strongest baseline by $+24.1$ pp on average (up to $+42.9$ pp), with the largest gains precisely where the baselines

¹Classification uses ground-truth entity annotations on SciERC and Re-DocRED, and GPT-5 (OpenAI, 2025) on MINE-1 with a spaCy NER sanity check (76.8% agreement); see Appendix H.

LLM	Method	MINE-1				SciERC				Re-DocRED			
		Single	Multi	Gap	Overall	Single	Multi	Gap	Overall	Single	Multi	Gap	Overall
GPT-4o	KGen	59.44	66.07	−6.63	65.05	62.74	68.36	−5.62	66.17	35.71	53.89	−18.18	49.33
	RAKG	63.08	77.68	−14.60	74.33	35.71	35.29	+0.42	35.46	37.50	56.29	−18.79	51.57
	HiReKG	91.28	94.03	−2.75	93.40	81.03	82.02	−0.99	81.65	80.36	80.24	+0.12	80.27
Qwen3-14B	KGen	45.93	62.72	−16.79	58.87	49.91	56.45	−6.54	53.91	50.00	45.51	+4.49	46.64
	RAKG	55.81	62.98	−7.17	61.33	43.50	46.86	−3.36	45.55	37.50	60.48	−22.98	54.71
	HiReKG	88.08	91.70	−3.62	90.87	76.95	79.93	−2.98	78.77	82.14	79.04	+3.10	79.82
Qwen3-8B	KGen	46.29	58.69	−12.40	55.85	53.73	51.93	+1.80	52.63	42.86	49.10	−6.24	47.53
	RAKG	62.50	77.16	−14.66	73.80	64.64	63.62	+1.02	64.02	41.07	59.88	−18.81	55.16
	HiReKG	77.03	85.29	−8.26	83.40	66.90	68.91	−2.01	68.13	60.71	65.27	−4.56	64.13

Table 1: MINE Score (recall, %) decomposed by entity count. **Single/Multi**: facts mentioning one or multiple entities. **Gap** = Single − Multi. **Overall**: full-dataset score. **Bold** marks the best per cell.

LLM	Method	MINE-1	SciERC	Re-DocRED
GPT-4o	KGen	90.23	83.44	85.05
	RAKG	57.38	61.11	64.01
	HiReKG	81.06	78.77	77.46
Qwen3-14B	KGen	93.96	93.73	95.70
	RAKG	81.38	78.01	76.24
	HiReKG	90.29	87.17	91.82
Qwen3-8B	KGen	91.83	90.38	94.74
	RAKG	80.90	73.99	73.79
	HiReKG	89.72	83.12	88.12

Table 2: Faithfulness Score (precision, %). KGen leads precision in every cell; HiReKG trails by 2–9 pp while delivering the largest recall gains (Table 1). **Bold** marks the best per cell.

collapse. The recall advantage is therefore not a generic uplift: it is concentrated on the class of facts the entity-first design undercaptures.

3.4 MINE and Faithfulness Scores

Tables 1 and 2 report overall recall and precision across the nine LLM–dataset configurations. HiReKG improves recall over the strongest baseline in every configuration, by +4.1 to +29.5 pp absolute (+6.4% to +55.7% relative). The gain is largest on MINE-1 and Re-DocRED with the mid-scale Qwen3-14B, and remains substantial on GPT-4o where the baselines are already at their strongest. RAKG and KGen alternate as the stronger baseline depending on dataset and model, but HiReKG outperforms both in every cell.

On Faithfulness, KGen retains the highest score in every configuration, consistent with its strict entity-pair anchoring. HiReKG trails KGen by 2.1 to 9.2 pp, with the gap narrowing on the open-weight models (2.1 to 7.3 pp). HiReKG substantially outperforms RAKG, whose Faithfulness falls as low as 57.4% on MINE-1 with GPT-4o.

HiReKG’s precision therefore sits between the two baselines while recovering the bulk of KGen’s recall gap, a net improvement for downstream tasks that value coverage.

3.5 Redundancy and Cost

We further measure graph redundancy via UIR and inference cost in total tokens per document. HiReKG achieves the highest overall UIR (0.71 vs. 0.68 for KGen and 0.59 for RAKG), producing graphs that are less redundant per triple, a property that matters for downstream tasks (Zhu et al., 2025). The cost overhead is bounded: HiReKG uses roughly 3× the tokens of KGen but less than half of RAKG. The additional tokens connect the scattered triples from property extraction, driving HiReKG’s recall and UIR gains. Detailed breakdowns are in Appendices E, F, and G.

4 Conclusion

We presented HiReKG, a text-to-KG extraction framework that decouples property extraction from entity-pair anchoring and restores graph connectivity through a dedicated object resolution module. Across nine LLM–dataset configurations, HiReKG achieves the highest recall in every cell while keeping precision within a narrow margin of the strongest entity-first baseline. A single-entity decomposition closes the loop: the systematic deficit on facts that describe a single entity in isolation, long latent in entity-first pipelines, is empirically identified and substantially recovered by HiReKG. The result demonstrates that the coverage ceiling of current text-to-KG systems is not intrinsic to the task but is an artifact of design choices that can be relaxed without sacrificing graph quality.

5 Limitations

Downstream evaluation. We evaluate HiReKG on intrinsic KG-quality metrics (recall, precision, redundancy) rather than on downstream tasks such as graph-based retrieval-augmented generation. A fair downstream comparison would require choosing retrieval methods per domain, which is a separate study in itself.

No multi-seed variance. The scores in Tables 1 and 2 are single-run point estimates; multi-seed confidence intervals were not feasible under our compute and API budget. We instead report per-document standard deviations in Appendix I, which capture LLM-judge variability across documents within each configuration.

Single-document scope. HiReKG produces one KG per input document. Multi-document KG construction is a harder, distinct problem that requires cross-document entity linking and conflict resolution between sources, and is outside the scope of this work.

Potential risks. HiReKG outputs are LLM-generated and may contain hallucinated triples; Faithfulness quantifies but does not eliminate this. MINE and Faithfulness also rely on LLM judges and inherit their biases. Downstream systems should verify triples against the source text before treating the graph as authoritative.

References

Gabor Angeli, Melvin Jose Johnson Premkumar, and Christopher D. Manning. 2015. [Leveraging linguistic structure for open domain information extraction](#). In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 344–354, Beijing, China. Association for Computational Linguistics.

Haonan Bian. 2025. Llm-empowered knowledge graph construction: A survey. *arXiv preprint arXiv:2510.20345*.

Wei Chen, Haoyu Huang, Zhiyu Zhang, Tianyi Wang, Youfang Lin, Liang Chang, and Huaiyu Wan. 2025. [Next-poi recommendation via spatial-temporal knowledge graph contrastive learning and trajectory prompt](#). *IEEE Transactions on Knowledge and Data Engineering*, 37(6):3570–3582.

Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt,

Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2025. [From local to global: A graph rag approach to query-focused summarization](#). *Preprint*, arXiv:2404.16130.

Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. 2024. Ragas: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th conference of the european chapter of the association for computational linguistics: system demonstrations*, pages 150–158.

Jiuzhou Han, Nigel Collier, Wray Buntine, and Ehsan Shareghi. 2024. [PiVe: Prompting with iterative verification improving graph-based generative capability of LLMs](#). In *Findings of the Association for Computational Linguistics ACL 2024*, pages 6702–6718, Bangkok, Thailand and virtual meeting. Association for Computational Linguistics.

Haoyu Huang, Chong Chen, Zeang Sheng, Yang Li, and Wentao Zhang. 2025. [Can LLMs be good graph judge for knowledge graph construction?](#) In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 10929–10948, Suzhou, China. Association for Computational Linguistics.

Yassir Lairgi, Ludovic Moncla, Rémy Cazabet, Khalid Benabdeslem, and Pierre Cléau. 2024. [itext2kg: Incremental knowledge graphs construction using large language models](#). *Preprint*, arXiv:2409.03284.

Yi Luan, Luheng He, Mari Ostendorf, and Hannaneh Hajishirzi. 2018. [Multi-task identification of entities, relations, and coreference for scientific knowledge graph construction](#). In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3219–3232, Brussels, Belgium. Association for Computational Linguistics.

Chuangtao Ma, Yongrui Chen, Tianxing Wu, Arijit Khan, and Haofen Wang. 2025. [Large language models meet knowledge graphs for question answering: Synthesis and opportunities](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 24578–24597, Suzhou, China. Association for Computational Linguistics.

Belinda Mo, Kyssen Yu, Joshua Kazdan, Proud Mpala, Lisa Yu, Charilaos Kanatsoulis, and Sanmi Koyejo. 2025. [Kggen: Extracting knowledge graphs from plain text with language models](#). In *Advances in Neural Information Processing Systems*, volume 38, pages 30092–30115. Curran Associates, Inc.

OpenAI. 2024. [GPT-4o System Card](#). *Preprint*, arXiv:2410.21276.

OpenAI. 2025. [Introducing GPT-5](#). GPT-5 model release and overview.

Qwen Team. 2024. [Qwen3-8B: Base Model](#). Part of the Qwen3 family; see Qwen3 Technical Report arXiv:2505.09388.

Qwen Team. 2025a. [Qwen3 Technical Report](#). *Preprint*, arXiv:2505.09388.

Qwen Team. 2025b. Qwen/qwen3-32b-fp8. <https://huggingface.co/Qwen/Qwen3-32B-FP8>. Hugging Face model card, accessed 2026-05-25.

Jiaqi Sun, Shiyu Qian, Zhangchi Han, Wei Li, Zelin Qian, Dingyu Yang, Jian Cao, and Guangtao Xue. 2025. Lkd-kgc: Domain-specific kg construction via llm-driven knowledge dependency parsing. *arXiv preprint arXiv:2505.24163*.

Qingyu Tan, Lu Xu, Lidong Bing, Hwee Tou Ng, and Sharifah Mahani Aljunied. 2022. [Revisiting DocRED - addressing the false negative problem in relation extraction](#). In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 8472–8487, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.

Yuan Yao, Deming Ye, Peng Li, Xu Han, Yankai Lin, Zhenghao Liu, Zhiyuan Liu, Lixin Huang, Jie Zhou, and Maosong Sun. 2019. DocRED: A large-scale document-level relation extraction dataset. In *Proceedings of ACL 2019*.

Wenhao Yu, Chenguang Zhu, Zaitang Li, Zhiting Hu, Qingyun Wang, Heng Ji, and Meng Jiang. 2022. [A survey of knowledge-enhanced text generation](#). *ACM Comput. Surv.*, 54(11s).

Hairong Zhang, Jiaheng Si, Guohang Yan, Boyuan Qi, Pinlong Cai, Song Mao, Ding Wang, and Botian Shi. 2025. [Rakg:document-level retrieval augmented knowledge graph construction](#). *Preprint*, arXiv:2504.09823.

Xiangrong Zhu, Yuexiang Xie, Yi Liu, Yaliang Li, and Wei Hu. 2025. Knowledge graph-guided retrieval augmented generation. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 8912–8924.

A End-to-End Walkthrough

To make the pipeline concrete, we trace HiReKG end-to-end on a single source paragraph. The blocks below show, in order, the source paragraph, subject extraction (Section 2.1), the raw (subject, relation, object) triples produced by property extraction (Section 2.2), the detection step that classifies each triple by how its object relates to the extracted entity set, the triples after object resolution (Section 2.3), and the final canonical triples after refinement. We omit some triples (marked . . .) for space; the full trace is available with the code release.

Source paragraph (2 chunks)

A butterfly’s life cycle has four stages: egg, larva, pupa, and adult. Female butterflies lay eggs on host plants, usually on the underside of leaves. After hatching, the larva, called a caterpillar, feeds on leaves and grows rapidly, shedding its skin through a process known as molting. The caterpillar then attaches to a stem and forms a chrysalis, inside which it undergoes metamorphosis into a butterfly.

The adult butterfly feeds on nectar through a tubular mouthpart called a proboscis, and pollinates flowers by transferring pollen between them. Adult butterflies typically live a few days to several weeks. Females then lay eggs on host plants, beginning the cycle again.

Stage 1 — Subject Extraction (25 subjects)

butterfly, life cycle, egg, larva, pupa, adult, female butterflies, host plants, leaves, caterpillar, skin, molting, stem, chrysalis, metamorphosis, adult butterfly, nectar, tubular mouthpart, proboscis, flowers, pollen, females, cycle, ...

Stage 2 — Property Extraction (raw triples, 20 total)

(life cycle, has, four stages: egg, larva, pupa, and adult)
 (female butterflies, lay, eggs on host plants)
 (female butterflies, lay eggs on, the underside of leaves)
 (larva, is called, a caterpillar)
 (larva, grows, rapidly)
 (larva, sheds, its skin)
 (caterpillar, undergoes, metamorphosis into a butterfly)
 (adult butterfly, feeds on, nectar)
 (adult butterfly, feeds through, a tubular mouthpart called a proboscis)
 (adult butterfly, live, a few days to several weeks)
 ...

Stage 3 — Detection (route each triple by entity count in object)

The object string is normalized and matched against the extracted entity set. In the displayed triples, [prefix] marks the text before a matched entity, {suffix} marks the text after, and «entity» marks a matched entity span.

Clean (no embedded entity, forwarded as-is)

(larva, grows, rapidly)
 (adult butterfly, live, a few days to several weeks)
 textbf
 Single embedded entity (deterministic prefix/suffix)

split)
(larva, is called, [a] «caterpillar»)
(larva, sheds, [its] «skin»)
(caterpillar, forms, [a] «chrysalis»)
(female butterflies, lay eggs on, [the
underside of] «leaves»)
(adult butterfly, transfers, «pollen»
{between them})

Multiple embedded entities (LLM-chosen strategy)

(life cycle, has, four stages: «egg»,
«larva», «pupa», and «adult»)
(female butterflies, lay, «eggs» on
«host plants»)
(adult butterfly, feeds through, a
«tubular mouthpart» called a «proboscis»)
(caterpillar, undergoes,
«metamorphosis» into a «butterfly»)
...

Stage 4 — Object Resolution (28 triples, grouped by strategy)

Clean (forwarded unchanged)

(larva, grows, rapidly)
(adult butterfly, live, a few days to several
weeks)

Single-entity — prefix folded into relation

(female butterflies, lay eggs on the underside
of, leaves)
(larva, is called, caterpillar)
(larva, sheds its, skin)
(caterpillar, attaches to, stem)
(caterpillar, forms, chrysalis)

Multi-entity — HUB (parallel list of objects)

(life cycle, has stage, egg)
(life cycle, has stage, larva)
(life cycle, has stage, pupa)
(life cycle, has stage, adult)

Multi-entity — CHAIN (sequential connectives)

(female butterflies, lay, eggs)
(eggs, laid on, host plants)
(caterpillar, undergoes, metamorphosis)
(metamorphosis, into, butterfly)

Multi-entity — NESTED (appositive / definitional)

(adult butterfly, feeds through a, tubular
mouthpart)
(tubular mouthpart, called a, proboscis)
...

Stage 5 — Refinement (canonicalization, 24 entities)

Refinement canonicalizes surface variants in the entity set. On this trace, the plural form eggs is folded into the singular egg, and the redundant second occurrence of host plants is removed, reducing the entity set from 25 to 24. Triples referencing the dropped surface forms are rewritten against the canonical entity, yielding the final knowledge graph.

A brief commentary on what each stage contributes. Subject extraction recovers the entities that anchor the document. Property extraction lifts the entity-pair constraint, allowing free-form objects: this is what preserves single-entity facts such as (larva, grows, rapidly) and (adult butterfly, live, a few days to several weeks), which entity-first pipelines drop by construction. The detection step then partitions raw triples into a clean set, a single-embedded-entity set that the prefix/suffix rule resolves deterministically, and a multi-embedded-entity set routed to the LLM for HUB, CHAIN, or NESTED decomposition. Object resolution applies these branches: a single raw triple about the four life-cycle stages becomes four has stage edges, and the appositive a tubular mouthpart called a proboscis becomes a two-hop chain. Refinement performs surface-form deduplication without changing semantics.

B Object Resolution: Worked Examples

Table 3 shows one worked example for each of the five resolution patterns introduced in Section 2.3. The first four rows are drawn from the end-to-end walkthrough in Appendix A; the NESTED row uses a constructed example, since the butterfly paragraph does not contain an appositive in the object position.

C Evaluation Prompts and Judge Configuration

Both MINE Score and Faithfulness Score are LLM-as-judge metrics. This appendix provides the verbatim prompts and the judge configuration used in all reported experiments.

Judge models. To avoid same-model bias between extractor and judge, the judge is matched by tier: **GPT-5** (OpenAI, 2025) judges all knowledge graphs extracted by GPT-4o, and **Qwen3-32B** (Qwen Team, 2025b) judges all knowledge graphs extracted by Qwen3-14B and Qwen3-8B. The judge model is held fixed across HiReKG and

Pattern	Raw triple (Stage 2 output)	Embedded entities	Resolved triples (Stage 4 output)
Clean	(larva, grows, rapidly)	<i>none</i>	(larva, grows, rapidly)
Single-entity (prefix)	(female butterflies, lay eggs on, the underside of leaves)	leaves	(female butterflies, lay eggs on the underside of, leaves)
Single-entity (suffix)	(adult butterfly, transfers, pollen between them)	pollen	(adult butterfly, transfers among themselves, pollen)
HUB	(life cycle, has, four stages: egg, larva, pupa, and adult)	egg, larva, pupa, adult	(life cycle, has stage, egg) (life cycle, has stage, larva) (life cycle, has stage, pupa) (life cycle, has stage, adult)
CHAIN	(female butterflies, lay, eggs on host plants)	eggs, host plants	(female butterflies, lay, eggs) (eggs, laid on, host plants)
NESTED [‡]	(treaty, signed in, Paris, the capital of France)	Paris, France	(treaty, signed in, Paris) (Paris, capital of, France)

Table 3: Worked examples for the five object-resolution patterns. The first four rows come from the butterfly walkthrough (Appendix A); the NESTED row is constructed. [‡] Constructed example; the butterfly paragraph does not contain an appositive in the object position.

both baselines within each tier, so the comparison is symmetric. Both judges are queried at high reasoning effort, with structured JSON output, and outputs that fail to parse as valid JSON are counted as judge errors and excluded from the denominator (see scoring formulas below).

MINE Score (recall). For each ground-truth atomic fact, top- $k=8$ KG nodes are retrieved by cosine similarity over all-MiniLM-L6-v2 embeddings of the fact against each node name. Each retrieved node is expanded by a two-hop traversal over outgoing and incoming edges, and every traversed edge (s, r, o) is serialized as the sentence " s r o ." (or " o r s ." for incoming edges). The resulting sentences are joined with single spaces into a flat context blob. We use the canonical retrieval routine from kg-gen (Mo et al., 2025) unchanged across all methods. The judge is then prompted with the context blob and the atomic fact:

MINE Score judge prompt (system + user)

System: You are an evaluator. Determine whether the provided context contains the information stated in the given correct answer. Think step-by-step internally, then respond with valid JSON of the form {"reasoning": "<brief reasoning>", "evaluation": 0 | 1}. Rules: evaluation = 1 if the context contains (supports) the correct answer, 0 otherwise. Evaluate on information content, not exact wording; paraphrases are fine. If the context is empty or irrelevant, return 0. Output ONLY the JSON object, no code fences, no extra keys.

User: Context: \n-\n{context}\n-\n\nCorrect

answer: \n-\n{correct_answer}\n-\n\nDoes the context contain the information stated in the correct answer? Respond with JSON.

The per-document MINE Score is the fraction of facts judged evaluation = 1, averaged across documents.

Faithfulness Score (precision). For each predicted triple (s, r, o) , a verbalization LLM converts the triple into a single self-contained English sentence (the *statement*) using the prompt below. The same retrieval embedder all-MiniLM-L6-v2 is then used to retrieve the top- $k=5$ most similar chunks of the source document as context. The judge receives the context and the statement and emits a SUPPORTED/NOT_SUPPORTED verdict with a required verbatim supporting span:

Faithfulness Score verbalization prompt

You convert a structured knowledge-graph triple (subject, predicate, object) into ONE self-contained English sentence that asserts the same fact.

Strict rules: Produce exactly ONE declarative sentence with no preamble. Preserve the subject and object strings verbatim; do not paraphrase, expand, abbreviate, change capitalization, or substitute synonyms. The predicate may be lightly inflected (tense, articles, auxiliary verbs) only as needed to form a grammatical sentence. Do not introduce information not present in the triple. Do not use pronouns; refer to the subject and object by their exact strings. Output only the sentence.

Input: subject: {subject}; predicate: {predicate}; object: {object}.

Faithfulness Score judge prompt

You are an entailment judge for knowledge-graph faithfulness evaluation. Given a SOURCE TEXT and a STATEMENT (verbalized from a predicted KG triple), decide whether the source text SUPPORTS the statement.

Verdicts: SUPPORTED if the source text explicitly entails the statement (paraphrases count, but the same proposition must be expressible from the source’s wording). NOT_SUPPORTED if the source either contradicts the statement or is simply silent on it.

Strict rules: Use ONLY the source text; ignore world knowledge and prior beliefs. Before deciding, attempt to identify a verbatim span from the source. If a span supports or contradicts the statement, copy it exactly; if the source is silent, set supporting_span to null. The reasoning field (1–2 sentences) must explicitly relate the cited span (or its absence) to the statement. Output a single strict JSON object with keys supporting_span, reasoning, verdict in that order. No code fences, no extra keys.

The verbalizer runs on GPT-5-mini (closed tier) or Qwen3-32B (open tier) at low reasoning effort. The per-document Faithfulness Score is

$$\text{Faithfulness} = \frac{n_{\text{supported}}}{n_{\text{total}} - n_{\text{malformed}} - n_{\text{judge_errors}}},$$

where $n_{\text{malformed}}$ counts triples whose verbalization failed schema validation and $n_{\text{judge_errors}}$ counts judge outputs that failed JSON parsing. The dataset score is the mean of per-document scores.

D Models and Hyperparameters

All extraction and judging calls use the same model snapshots across HiReKG, KGGen, and RAKG, so observed differences reflect pipeline design rather than the choice of LLM. The extractors are gpt-4o-2024-08-06 (matching the snapshot in Mo et al., 2025) and the FP8-quantized open-source models Qwen/Qwen3-14B-FP8 and Qwen/Qwen3-8B-FP8. The judge is gpt-5-2025-08-07 for the GPT-4o runs and Qwen/Qwen3-32B-FP8 for the Qwen3 runs; the latter is held distinct from both Qwen3 extractors to avoid same-model bias.

Decoding and chunking. Table 4 summarizes per-method decoding and chunking. Top- p and seed are left unset; streaming is disabled so responses return as complete JSON. Chunking is part of each published pipeline and is not normalized across methods.

Pipeline thresholds. Every numerical threshold used inside HiReKG and the evaluation pipelines is listed below.

Setting	HiReKG	KGGen	RAKG
Temperature	0.0 / 0.2	0.0	0.0
Max output tokens	13,000	10,000	13,000
Max retries	3	3	3
Chunk unit	sentences	characters	sentences
Chunk size	3–4 sent. [†]	3000 chars	1 sent.
Sentencizer	en_core_web_sm	—	native

Table 4: Decoding and chunking configuration.

[†] HiReKG uses 3 sentences per chunk in OpenAI Batch mode (temperature 0.0) and 4 sentences per chunk in non-batched mode (temperature 0.2).

HiReKG internals. Entity refinement mode llm (batch) with WordNetLemmatizer normalization. Semantic linking uses BAAI/bge-m3 embeddings with cluster_distance_threshold= 0.99 and min_cluster_size= 2.

Evaluation pipelines. **MINE Score** retrieves the top- $k=8$ KG nodes by all-MiniLM-L6-v2 cosine similarity and expands to a 2-hop subgraph (defaults from Mo et al., 2025). **Faithfulness Score** retrieves the top- $k=5$ most similar source chunks for each verbalized triple, also using all-MiniLM-L6-v2. **UIR** embeds verbalized triples with all-MiniLM-L6-v2 and clusters by pairwise BERTScore.

Baselines. KGGen deduplication uses semhash_threshold= 0.95. RAKG retrieval uses BAAI/bge-m3 embeddings with similarity_threshold= 0.60 and retrieval_top_k= 5.

D.1 Baseline Implementations

We use the official open-source implementations of both baselines without modification to their core logic.

D.2 Serving Stack and Hardware

The two Qwen3 models are served locally with vLLM using FP8 quantization. The GPT-4o and GPT-5 calls are issued through the OpenAI API; GPT-4o extraction calls additionally use the OpenAI Batch API for cost efficiency on the larger Qwen-comparable runs. Experiments are distributed across two servers: one with a single NVIDIA A100 80GB GPU and one with a single NVIDIA H100 80GB GPU. Different datasets are run on whichever server is free at the time, so the assignment of dataset to GPU type is not informative; the extraction LLMs themselves are deterministic enough at temperature 0.0–0.2 that the GPU model

does not materially affect the reported scores.

E Unique Information Ratio

Metric. The Unique Information Ratio (UIR) measures redundancy among predicted triples. Each triple (s, r, o) is verbalized into a short sentence and embedded with all-MiniLM-L6-v2; pairs whose embedding similarity exceeds a threshold are clustered together, so that paraphrased triples expressing the same fact collapse into a single cluster. UIR is defined as

$$\text{UIR} = \frac{n_{\text{clusters}}}{n_{\text{triples}}},$$

where higher values indicate that every triple contributes a distinct fact and lower values indicate redundancy. A graph in which every triple is unique has $\text{UIR} = 1.0$; a graph whose triples collapse into half as many distinct facts has $\text{UIR} = 0.5$.

Per-dataset results. Table 6 reports UIR for each method on each dataset, averaged across the three extractor LLMs (GPT-4o, Qwen3-14B, Qwen3-8B).

Discussion. HiReKG achieves the highest overall UIR (0.71), ahead of KGGen (0.68) and RAKG (0.59). The advantage is largest on Re-DocRED (+10.9 pp over KGGen), where the longer Wikipedia-style documents give entity-first methods more opportunities to emit paraphrased triples, and on SciERC (+2.9 pp over KGGen). KGGen edges HiReKG on MINE-1 (0.69 vs. 0.65, a 3.7 pp gap), reflecting KGGen’s strict entity-pair anchoring on a dataset where the short LLM-generated documents naturally yield fewer redundant triples to begin with. RAKG trails on every dataset, consistent with its much higher absolute triple counts (38,249 raw triples across all configurations, against 25,925 for HiReKG and 16,379 for KGGen) inflating the denominator faster than the cluster count.

F Inference Cost

Table 7 reports the average per-document token cost of each method, averaged across the three extractor LLMs (GPT-4o, Qwen3-14B, Qwen3-8B). Counts are total tokens billed to the inference API, decomposed into input and output.

HiReKG sits between the two baselines on every dataset. Averaging across datasets, HiReKG uses $2.9\times$ the tokens of KGGen (35.8k vs. 12.2k per

document) but only $0.40\times$ the tokens of RAKG (35.8k vs. 90.0k). The per-dataset ratio against KGGen is stable ($2.6\text{--}3.0\times$), confirming that the cost overhead is constant across text styles rather than concentrated on any one dataset. Against RAKG the gap is widest on Re-DocRED, where HiReKG uses $0.34\times$ the tokens; this is the dataset where RAKG’s per-document retrieval cost is highest. Output tokens dominate the cost of all three methods (62–75% of the total), so the differences in total cost track the differences in generation volume rather than retrieval volume. The output tokens also include the tokens used for thinking.

G Object Resolution Statistics

Table 5 reports the operational statistics of the object resolution module across all nine LLM–dataset configurations. **Rep. %** is the fraction of raw extractor triples that the module rewrites; **Fail %** is the fraction whose rewrite fails validation and reverts to the original; **Yield %** is the net change in triple count after resolution. The remainder of triples in each cell pass through unchanged.

The repair rate is tightly bounded across configurations (57.3–74.4%), showing that object resolution fires at a stable rate independent of dataset or extractor scale. The clean rate (fraction of triples forwarded unchanged) ranges from 16.1% to 42.4%, confirming that the module is adaptive: well-formed triples are forwarded as-is rather than rewritten. Yield is largest where GPT-4o produces dense compound objects that the module decomposes (+38.5% on Re-DocRED, +37.8% on MINE-1) and smallest where Qwen3-14B’s higher fail rate (9.5–13.4%) reverts more triples to their unprocessed form (+9.1% on Re-DocRED). The elevated Qwen3-14B fail rate is concentrated entirely in this configuration and does not appear in 8B scale; we attribute it to occasional JSON-schema deviations in the mid-scale model’s output, which the validation layer correctly catches before downstream stages.

H Single/Multi-Entity Classification

The single-entity decomposition reported in Section 3.3 requires classifying every atomic fact in the recall ground truth as *single-entity* (mentioning exactly one of the document’s entities) or *multi-entity* (mentioning two or more). The classifier differs by dataset because the available annotations differ.

LLM	MINE-1			SciERC			Re-DocRED		
	Rep. %	Fail %	Yield %	Rep. %	Fail %	Yield %	Rep. %	Fail %	Yield %
GPT-4o	72.0	1.5	+37.8	57.3	0.3	+17.5	73.8	0.4	+38.5
Qwen3-14B	63.9	13.4	+13.3	68.7	11.7	+14.5	74.4	9.5	+9.1
Qwen3-8B	64.9	2.4	+11.1	68.8	2.2	+13.2	66.9	2.4	+12.2

Table 5: Object resolution statistics across nine LLM–dataset configurations. Aggregated across configurations (triple-weighted), the module repairs 67.0% of raw triples, fails on 5.6%, and increases the final triple count by +20.4%.

Method	MINE-1	SciERC	Re-DocRED	Overall
KGGen	0.69	0.80	0.54	0.68
RAKG	0.58	0.69	0.49	0.59
HiReKG	0.65	0.83	0.65	0.71

Table 6: Unique Information Ratio (UIR), averaged across GPT-4o, Qwen3-14B, and Qwen3-8B for each (method, dataset) pair. **Bold** marks the best per column.

Method	Tokens	MINE-1	SciERC	Re-DocRED
KGGen	Input	4,829	2,107	2,161
	Output	16,721	3,864	6,959
	Total	21,550	5,971	9,120
RAKG	Input	65,889	6,803	29,140
	Output	99,079	17,877	51,336
	Total	164,968	24,680	80,476
HiReKG	Input	22,150	4,653	8,808
	Output	42,382	10,936	18,534
	Total	64,532	15,589	27,342

Table 7: Average tokens per document, averaged across the three extractor LLMs. Input/output split exposes which methods are retrieval-heavy (input) versus generation-heavy (output).

SciERC and Re-DocRED. Both datasets provide gold-annotated entity sets for each document. We apply deterministic string matching: each annotated entity is searched in the fact text with case-insensitive, whitespace-normalized, word-boundary-aware regex matching. Overlapping matches are deduplicated by retaining only the longest non-overlapping span, so that “Microsoft Windows” is not double-counted as “Microsoft” plus “Windows”. A fact with one matched entity is labeled single-entity, with two or more matched entities multi-entity, and with zero matched entities ambiguous (the latter folded into multi-entity for analysis, matching the strictest possible single-entity definition).

MINE-1. MINE-1 atomic facts do not come with annotated entity sets, so deterministic match-

ing is not available. We classify each fact with GPT-5 (OpenAI, 2025) using a zero-shot prompt that asks the model to count the entities mentioned in the fact and emit one of single-entity, multiple-entity, or ambiguous. The prompt instructs GPT-5 to treat both proper nouns (e.g., “Marie Curie”) and common-noun entities (e.g., “butterfly”, “chrysalis”) as entities, since MINE-1’s LLM-generated explanatory prose contains few proper nouns. Ambiguous facts are again folded into multi-entity for analysis.

Validation against spaCy NER. We validated the GPT-5 labels against spaCy NER (en_core_web_sm), applying the same count-based classification rule to spaCy’s entity spans. Across 1,500 MINE-1 facts, GPT-5 and spaCy agreed on 76.8% of labels (Cohen’s $\kappa = 0.32$). The disagreement is asymmetric and concentrated where expected: spaCy systematically under-recognizes common-noun entities that off-the-shelf NER models are not trained to tag (“butterfly”, “caterpillar”, “chrysalis”), but agrees strongly with GPT-5 on facts containing proper nouns ($P=0.84$, $R=0.86$ on multi-entity facts). Our analysis uses the GPT-5 labels because they more accurately reflect MINE-1’s actual entity structure; the residual disagreement on common-noun-heavy facts is the expected behavior of off-the-shelf NER rather than a labeling error.

Fact distribution. Table 9 reports the resulting single-entity and multi-entity counts per dataset (ambiguous facts folded into multi-entity per the convention above). SciERC is the most balanced of the three; MINE-1 and Re-DocRED are similarly skewed toward multi-entity, with single-entity facts forming roughly a quarter of each dataset. Re-DocRED’s smaller absolute count (56 single-entity facts) reflects its 15-document scope and is sufficient for the per-cell averages reported in Section 3.3.

LLM	Method	MINE-1	SciERC	Re-DocRED
GPT-4o	KGGen	65.88 $\pm$ 18.83	66.23 $\pm$ 18.18	49.21 $\pm$ 22.19
	RAKG	74.33 $\pm$ 16.37	33.34 $\pm$ 20.38	51.43 $\pm$ 15.40
	HiReKG	93.40 $\pm$ 7.51	80.15 $\pm$ 13.94	80.19 $\pm$ 13.83
Qwen3-14B	KGGen	58.87 $\pm$ 21.93	53.81 $\pm$ 20.34	46.70 $\pm$ 23.87
	RAKG	61.33 $\pm$ 17.76	45.58 $\pm$ 18.85	54.79 $\pm$ 13.22
	HiReKG	90.87 $\pm$ 9.17	78.74 $\pm$ 15.97	79.81 $\pm$ 10.34
Qwen3-8B	KGGen	54.73 $\pm$ 22.08	52.52 $\pm$ 18.63	47.36 $\pm$ 16.65
	RAKG	73.80 $\pm$ 15.12	63.97 $\pm$ 18.64	55.14 $\pm$ 14.61
	HiReKG	83.40 $\pm$ 11.17	68.10 $\pm$ 17.43	64.16 $\pm$ 17.73

Table 8: MINE Score (%) reported as mean $\pm$ standard deviation across documents within each (method, dataset, LLM) configuration.

Dataset	Single	Multi	Total
MINE-1	344 (22.9%)	1,156 (77.1%)	1,500
SciERC	577 (38.9%)	907 (61.1%)	1,484
Re-DocRED	56 (25.1%)	167 (74.9%)	223

Table 9: Distribution of atomic facts across single-entity and multi-entity classes per dataset, after folding the small ambiguous bucket into multi-entity (Section 3.3).

I Per-Document Variance

Table 8 reports the per-document mean and standard deviation of MINE Score for each method, dataset, and extractor LLM. Each value is computed across the documents in the dataset’s evaluation split, after running the full extraction and judging pipeline once per document.

Variance reduction. Beyond the recall gains reported in the main paper, HiReKG also exhibits consistently lower per-document variance than either baseline. Averaged across the nine configurations, HiReKG’s standard deviation is 13.0, compared with 16.7 for RAKG and 20.3 for KGGen. The reduction is largest on MINE-1 with GPT-4o (-8.9 std relative to the strongest baseline) and holds in 8 of 9 configurations; the single exception is Re-DocRED with Qwen3-8B ($+3.1$ std), which is also the only configuration where HiReKG’s mean score does not exceed the strongest baseline by a wide margin. Lower variance means HiReKG’s recall advantage is distributed across documents rather than driven by a small number of high-scoring outliers, which makes the headline gains in Section 3.4 more reliable as a per-document expectation.

What is and is not measured. The standard deviations above capture variation *across documents*

within a single run of each pipeline. They do not capture variation *across LLM rollouts* on the same document, which would require multi-seed runs that were not feasible under our compute budget. We note this as a limitation.

J Artifacts and Licenses

MINE-1, KGGen (Mo et al., 2025), Re-DocRED (Tan et al., 2022), DocRED (Yao et al., 2019), RAKG (Zhang et al., 2025), and spaCy are released under the MIT License. SciERC (Luan et al., 2018) is released under CC BY 4.0. Qwen3-14B and Qwen3-8B (Qwen Team, 2025a, 2024) are released under Apache 2.0; GPT-4o (OpenAI, 2024) is accessed via the OpenAI API. All use is consistent with the respective licenses and intended for research. HiReKG is released under the MIT License.